PYTHON MINI PROJECT

INCLUSIVE LEARNING-MATERIAL SELECTION UNDER BUDGET

Optimal resource selection using 0/1 Knapsack Dynamic Programming

Department of Computer Science and Engineering

Abstract

- Educational institutions often have limited budgets for learning materials.
- This project selects materials that maximize accessibility benefit without exceeding the budget.
- The 0/1 Knapsack problem is implemented in Python using Dynamic Programming.
- Greedy is used for comparison and Brute Force verifies small test cases.

Introduction

- Inclusive education aims to make learning resources accessible to every student.
- Braille kits, audio resources, and accessible software have different costs and benefits.
- A limited budget makes selecting the right combination an optimization problem.
- This project models the task as a 0/1 Knapsack problem.

Problem Statement & Objectives

- Problem: Select learning materials within a fixed budget to maximize total accessibility benefit.
- Accept material names, costs, benefits, and total budget.
- Find the optimal selection using Dynamic Programming.
- Compare with Greedy and verify small cases using Brute Force.

Algorithms Used

1. Dynamic Programming

Builds a table of best benefits for each item and budget. Guarantees the optimal 0/1 Knapsack result.

2. Greedy Algorithm

Sorts materials by benefit-to-cost ratio and selects items that fit. Used as a comparison baseline.

3. Brute Force

Checks all possible subsets and finds the best feasible benefit. Suitable for small inputs.

0/1 Knapsack – Dynamic Programming

For each material i and available budget w:

If cost[i] ≤ w:

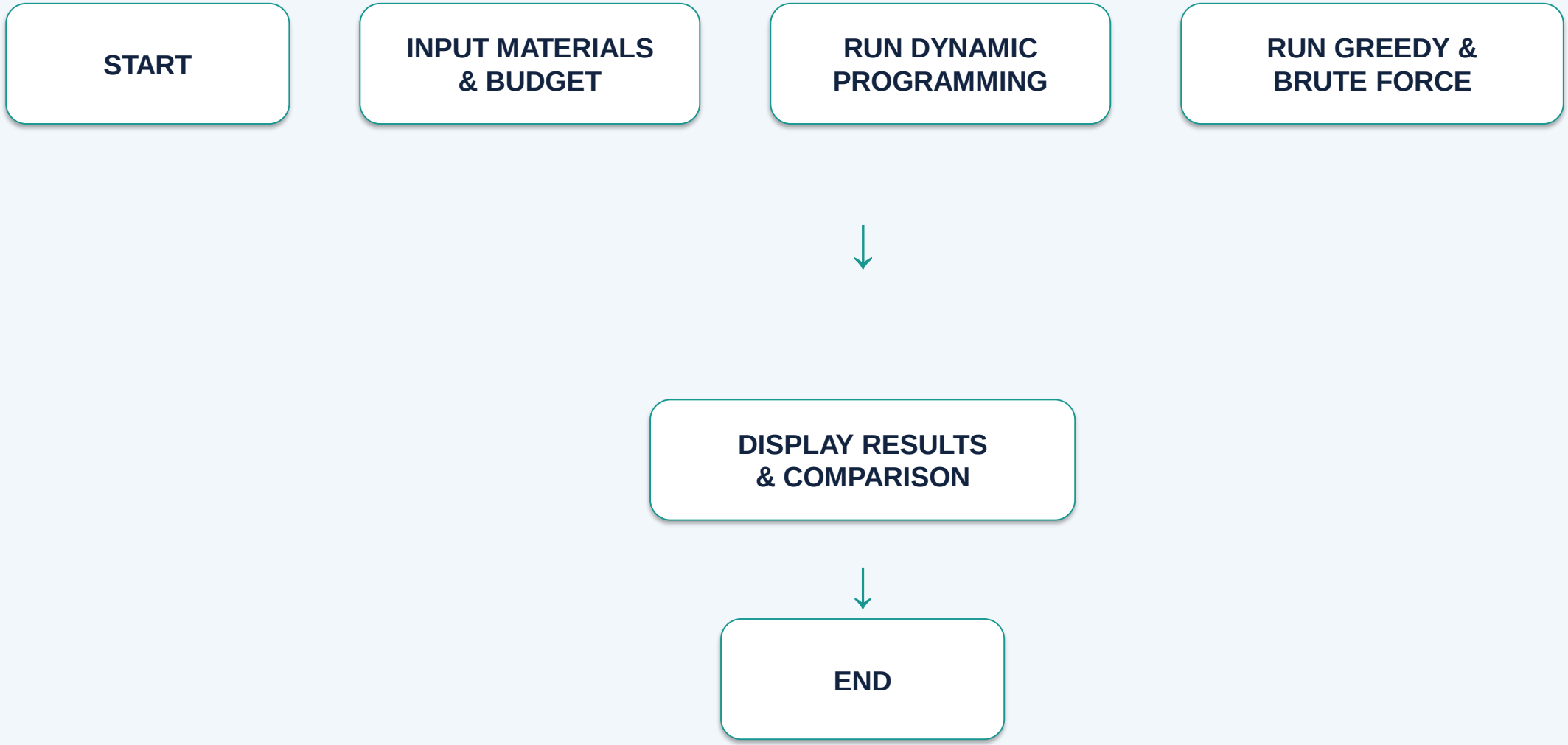
$$DP[i][w] = \max(benefit[i] + DP[i-1][w-cost[i]], DP[i-1][w])$$

Otherwise:

$$DP[i][w] = DP[i-1][w]$$

The final table entry gives the maximum accessibility benefit within the budget.

System Flowchart



Python Implementation

- Language: Python 3.
- Main functions: knapsack(), greedy(), brute_force(), and main().
- Lists store material names, costs, and benefits.
- Nested loops build the Dynamic Programming table.
- Sorting arranges materials by benefit-to-cost ratio.

Methodology

- Collect material names, costs, benefits, and total budget.
- Validate inputs, including positive costs and non-negative budget.
- Run Dynamic Programming to calculate the optimal selection.
- Run Greedy and Brute Force algorithms for comparison.
- Display selected materials, total cost, benefits, and comparison.

Advantages

- Finds the optimal combination within the budget.
- Considers both material cost and accessibility benefit.
- Compares Dynamic Programming with Greedy selection.
- Brute Force verifies small test cases.
- Supports informed allocation of educational resources.

Conclusion

- The project models material selection as a 0/1 Knapsack problem.
- Python Dynamic Programming finds the maximum benefit within budget.
- Greedy provides comparison but may not be optimal.
- Brute Force verifies the DP result for small inputs.
- The project supports inclusive education resource planning.

Future Enhancements

- Develop a graphical user interface.
- Store material information in a database.
- Add accessibility categories and selection reports.
- Support larger datasets and advanced resource allocation.

Sample Results

Sample input: Total budget = 300

Learning Material	Cost	Benefit
Braille Learning Kit	200	90
Audio Learning Resources	150	80
Accessible Learning Software	100	85

Dynamic Programming: 175 | Greedy: 165 | Brute Force: 175

DP and Brute Force agree, verifying the optimal result for this sample.

Add an actual screenshot of your Python program output before submission.

Complexity Analysis

Dynamic Programming

Time: $O(n \times W)$
Space: $O(n \times W)$
n = number of materials
W = budget capacity

Greedy

Time: $O(n \log n)$
Sorting by benefit-to-cost ratio is the main operation.

Brute Force

Time: $O(n \times 2^n)$
Checks every subset; suitable for small inputs.

Applications

- Schools and colleges
- Inclusive education institutions
- Accessibility resource planning
- Educational budget management

THANK YOU